

LAPORAN TUGAS BESAR 1
IF2224 Teori Bahasa Formal dan Automata
Semester II 2025/2026

Arion Compiler Milestone 2 - Syntax Analysis



Stevanus Agustaf Wongso	13524020
Renuno Yuqa Frinardi	13524080
Valentino Daniel Kusumo	13524104
Michael James Liman	13524106

Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2026

DAFTAR ISI

DAFTAR ISI.....	1
BAB I	
DASAR TEORI.....	2
1. Context Free Language (CFL).....	2
2. Context Free Grammar (CFG).....	2
3. Parser.....	3
4. Parse Tree.....	3
5. Recursive Descent Parser.....	4
BAB II	
PERANCANGAN DAN IMPLEMENTASI.....	5
1. Perancangan.....	5
1.1 Gambaran Umum Sistem.....	5
1.2 Perancangan Grammar.....	5
1.3 Production Rules.....	5
1.4 Strategi Parsing.....	8
2. Implementasi.....	9
2.1 Arsitektur File.....	9
2.2 Struktur Data.....	11
2.3 Implementasi Parser.....	12
Parser.....	12
DeclarationParser.....	14
StatementSubprogramParser.....	16
ExpressionParser.....	18
BAB III	
PENGUJIAN.....	21
1. Test Case 1.....	21
2. Test Case 2.....	22
3. Test Case 3.....	25
4. Test Case 4.....	29
5. Test Case 5.....	32
BAB IV	
KESIMPULAN DAN SARAN.....	36
1. Kesimpulan.....	36
2. Saran.....	36
LAMPIRAN.....	38
REFERENSI.....	39

BAB I

DASAR TEORI

1. Context Free Language (CFL)

Context-Free Language adalah bahasa formal yang dibentuk oleh *Context-Free Grammar*. CFL umumnya dapat diterima oleh sebuah *Pushdown Automata* dan penting dalam proses pengembangan bahasa pemrograman & compiler. CFL umumnya memiliki beberapa properti:

- a. **Klosur:** CFL bersifat klosur dalam beberapa operasi, artinya sebuah CFL jika dilakukan operasi tertentu akan menghasilkan CFL juga. Tetapi ini tidak berlaku dalam beberapa operasi seperti, Intersection, Complement, Subset, etc.
- b. **Decision Property:**
 - Emptiness, bisa ditentukan apakah CFL kosong atau tidak
 - Finiteness, bisa ditentukan apakah CFL memiliki banyak string hingga
 - Membership, bisa ditentukan apakah string termasuk ke dalam CFL
 - Equivalence, tidak bisa selalu ditentukan apakah dua buah CFL sama persis
 - Universality, tidak bisa ditentukan
 - Inclusion, tidak bisa ditentukan

Terdapat sebuah lemma khusus yang pasti dipenuhi oleh seluruh CFL. Lemma ini ditujukan untuk membuktikan apakah sebuah Language merupakan CFL. Nama lemma ini merupakan **Pumping Lemma**. Dinyatakan bahwa untuk setiap CFL L, pasti terdapat dua buah substring yang bisa di 'pompa' sebanyak hingga kali dan hasilnya tetap merupakan L. Lebih jelasnya.

2. Context Free Grammar (CFG)

Context Free Grammar (CFG) adalah salah satu jenis formal grammar yang digunakan untuk mendefinisikan struktur sintaksis suatu bahasa pemrograman. CFG disebut "context-free" karena setiap aturan produksi hanya bergantung pada non-terminal itu sendiri, tidak bergantung pada simbol-simbol di sekitarnya (konteksnya). Hal ini berarti suatu non-terminal dapat diturunkan dengan cara yang sama di manapun ia muncul dalam sebuah derivasi.

CFG banyak digunakan dalam perancangan parser karena struktur bahasa pemrograman umumnya dapat direpresentasikan secara hierarkis dan rekursif, yang merupakan karakteristik utama CFG. Hasil dari proses parsing menggunakan CFG adalah sebuah Parse Tree yang merepresentasikan struktur hierarkis program sesuai dengan aturan grammar yang telah didefinisikan.

Grammar secara formal didefinisikan sebagai 4-tuple atau 4 komponen utama.

$$G = (V_N, V_T, P, S)$$

- **V_N (Non-terminal symbols / Variabel):** Himpunan hingga dari simbol-simbol non-terminal atau variabel. Simbol ini adalah simbol yang dapat digantikan atau diturunkan dan dipresentasikan dengan huruf kapital.
- **V_T (Terminal symbols):** Himpunan hingga dari simbol-simbol terminal. Simbol ini adalah simbol yang tidak dapat digantikan lagi atau simbol akhir dan direpresentasikan dengan huruf kecil, angka, atau simbol khusus.
- **P (Production Rules):** Himpunan hingga dari aturan produksi. Aturan ini menentukan bagaimana simbol non-terminal dapat diganti oleh rangkaian simbol terminal dan/atau non-terminal.
- **S (Start Symbol):** . Simbol non-terminal yang menjadi awal mula *derivation* dari semua string dalam bahasa tersebut.

3. Parser

Parsing atau analisis syntax merupakan proses dalam menganalisa token-token untuk menentukan susunan struktur dari sebuah bahasa yang sesuai dengan grammar yang membentuknya. Parser umumnya menghasilkan sebuah *parse tree* yang akan menggambarkan secara visual bagaimana token token saling berkorelasi antara satu sama lain. Selain itu, proses parser juga memastikan agar seluruh token tetap memenuhi aturan grammar dari bahasa tersebut.

Terdapat beberapa tujuan dari parser, antara lain:

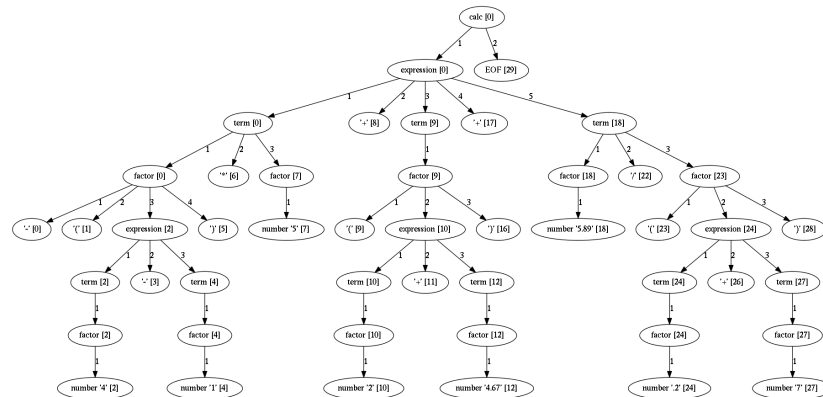
1. Melakukan analisis konteks
2. Membantu analisis konteks
3. Membuat bentuk perantara
4. Membentuk error message
5. Melakukan koreksi error

Selain itu, parsing juga dibagi menjadi ke dua jenis, yaitu:

1. Top-Down Parsing, parsing yang dilakukan dari root (start symbol) ke daun (end symbol)
2. Bottom-Up Parsing, parsing yang dilakukan dari daun (end symbol) ke root (start symbol)

4. Parse Tree

Parse Tree merupakan suatu representasi dalam bentuk tree dari *derivations* yang menyajikan simbol dari *terminal string* digabungkan menjadi *substring* yang merupakan variabel yang ada pada *grammar* dari suatu language. Pada compiler, parse tree berperan sebagai translator dari *source program* menjadi *executable code* dengan memfasilitasi *recursive functions* dalam proses translasi.



Source: https://textx.github.io/Arpeggio/latest/parse_trees

Sebuah fungsi **non-terminal** adalah fungsi yang membentuk root atau cabang, sedangkan fungsi **terminal** adalah fungsi yang membentuk daun pada parse tree.

5. Recursive Descent Parser

Recursive Descent Parser merupakan parser berjenis top-down parser yang memproses input token dengan beberapa set fungsi rekursif, di mana setiap fungsi rekursif merupakan aturan grammar dari bahasanya itu sendiri. Parser berjenis ini merupakan salah satu parser yang paling mudah untuk diimplementasi dan cocok untuk parsing berjenis LL(1), di mana proses parsing dapat dilakukan dengan hanya menyimpan satu lookahead token.

BAB II

PERANCANGAN DAN IMPLEMENTASI

1. Perancangan

1.1 Gambaran Umum Sistem

Program ini merupakan lanjutan dari tahap sebelumnya, yaitu *Lexical Analysis* yang menghasilkan daftar *tokens*. *Token - token* tersebut akan menjadi *input* pada tahap ini, yaitu *Syntax Analysis (Parsing)* yang akan menghasilkan struktur data hierarkis berupa *Parse tree*.

Alur kerja program adalah sebagai berikut:

- **Input:** Program menerima daftar token yang telah dihasilkan pada tahap *Lexical Analysis*.
- **Proses:** Parser memproses urutan token menggunakan aturan tata bahasa (*Context-Free Grammar*) dengan algoritma *Recursive Descent* untuk memastikan bahwa susunan token tersebut legal secara struktur bahasa pemrograman.
- **Output:** Hasil akhirnya adalah sebuah *Parse Tree* yang merepresentasikan struktur hierarkis program, di mana *root* merepresentasikan keseluruhan program dan daun merepresentasikan *token* individual dari input.

Secara garis besar, alur program adalah sebagai berikut:

Source code (.txt) → Lexer → List of tokens → Parser → Parse Tree

1.2 Perancangan Grammar

Dalam implementasi program ini, 4 komponen grammar didefinisikan sebagai berikut:

- **V_N :** Seluruh konstruksi sintaksis yang diturunkan oleh parser, ditulis dengan tanda kurung sudut $\langle \rangle$. Contoh: $\langle \text{program} \rangle$, $\langle \text{statement} \rangle$.
- **V_T :** Seluruh token yang dihasilkan oleh lexer dan tidak diturunkan lebih lanjut. Contoh: *programsy*, *ident*, *intcon*, *realcon*, *charcon*, *string*.
- **P:** Seluruh aturan produksi dalam *parser*, yang akan dijelaskan lebih lengkap pada bagian [berikutnya](#).
- **S:** $\langle \text{program} \rangle$, yaitu non-terminal awal yang merepresentasikan keseluruhan program.

1.3 Production Rules

Berikut adalah seluruh aturan produksi yang digunakan dalam parser:

Program Utama

$\langle \text{program} \rangle \rightarrow \langle \text{program-header} \rangle + \langle \text{declaration-part} \rangle + \langle \text{compound-statement} \rangle + \text{period}$
 $\langle \text{program-header} \rangle \rightarrow \text{programsy} + \text{ident} + \text{semicolon}$

Declaration Part

$\langle \text{declaration-part} \rangle \rightarrow \langle \text{const-declaration} \rangle^* + \langle \text{type-declaration} \rangle^* + \langle \text{var-declaration} \rangle^* + \langle \text{subprogram-declaration} \rangle^*$

Const Declaration

$\langle \text{const-declaration} \rangle \rightarrow \text{constsy} + (\text{ident} + \text{eql} + \langle \text{constant} \rangle + \text{semicolon}) + \langle \text{constant} \rangle \rightarrow \text{charcon} \mid \text{string} \mid [(\text{plus} \mid \text{minus})? + (\text{ident} \mid \text{intcon} \mid \text{realcon})]$

Type Declaration

$\langle \text{type-declaration} \rangle \rightarrow \text{typesy} + (\text{ident} + \text{eql} + \langle \text{type} \rangle + \text{semicolon}) + \langle \text{type} \rangle \rightarrow \text{ident} \mid \langle \text{array-type} \rangle \mid \langle \text{range} \rangle \mid \langle \text{enumerated} \rangle \mid \langle \text{record-type} \rangle$
 $\langle \text{array-type} \rangle \rightarrow \text{arraysy} + \text{lbrack} + (\langle \text{range} \rangle \mid \text{ident}) + \text{rbrack} + \text{ofsy} + \langle \text{type} \rangle$
 $\langle \text{range} \rangle \rightarrow \langle \text{constant} \rangle + \text{period} + \text{period} + \langle \text{constant} \rangle$
 $\langle \text{enumerated} \rangle \rightarrow \text{lparent} + \text{ident} + (\text{comma} + \text{ident})^* + \text{rparent}$
 $\langle \text{record-type} \rangle \rightarrow \text{recordsy} + \langle \text{field-list} \rangle + \text{endsy}$
 $\langle \text{field-list} \rangle \rightarrow \langle \text{field-part} \rangle + (\text{semicolon} + \langle \text{field-part} \rangle)^*$
 $\langle \text{field-part} \rangle \rightarrow \langle \text{identifier-list} \rangle + \text{colon} + \langle \text{type} \rangle$

Var Declaration

$\langle \text{var-declaration} \rangle \rightarrow \text{varsy} + (\langle \text{identifier-list} \rangle + \text{colon} + \langle \text{type} \rangle + \text{semicolon}) + \langle \text{identifier-list} \rangle \rightarrow \text{ident} + (\text{comma} + \text{ident})^*$

Subprogram Declaration

$\langle \text{subprogram-declaration} \rangle \rightarrow \langle \text{procedure-declaration} \rangle \mid \langle \text{function-declaration} \rangle$
 $\langle \text{procedure-declaration} \rangle \rightarrow \text{proceduresy} + \text{ident} + (\langle \text{formal-parameter-list} \rangle)? + \text{semicolon} + \langle \text{block} \rangle + \text{semicolon}$
 $\langle \text{function-declaration} \rangle \rightarrow \text{functionsy} + \text{ident} + (\langle \text{formal-parameter-list} \rangle)? + \text{colon} + \text{ident} + \text{semicolon} + \langle \text{block} \rangle + \text{semicolon}$
 $\langle \text{block} \rangle \rightarrow \langle \text{declaration-part} \rangle + \langle \text{compound-statement} \rangle$
 $\langle \text{formal-parameter-list} \rangle \rightarrow \text{lparent} + \langle \text{parameter-group} \rangle + (\text{semicolon} + \langle \text{parameter-group} \rangle)^* + \text{rparent}$

$\langle \text{parameter-group} \rangle \rightarrow \langle \text{identifier-list} \rangle + \text{colon} + (\text{ident} \mid \langle \text{array-type} \rangle)$

Compound Statement

$\langle \text{compound-statement} \rangle \rightarrow \text{beginsy} + \langle \text{statement-list} \rangle + \text{endsy}$
 $\langle \text{statement-list} \rangle \rightarrow \langle \text{statement} \rangle + (\text{semicolon} + \langle \text{statement} \rangle)^*$

Statement

$\langle \text{statement} \rangle \rightarrow \langle \text{assignment-statement} \rangle \mid \langle \text{if-statement} \rangle \mid \langle \text{case-statement} \rangle$
 $\mid \langle \text{while-statement} \rangle \mid \langle \text{repeat-statement} \rangle \mid \langle \text{for-statement} \rangle \mid$
 $\langle \text{procedure/function-call} \rangle \mid \epsilon$

Assignment & Variable

$\langle \text{assignment-statement} \rangle \rightarrow \langle \text{variable} \rangle + \text{becomes} + \langle \text{expression} \rangle$
 $\langle \text{variable} \rangle \rightarrow \text{ident} + (\langle \text{component-variable} \rangle)^*$
 $\langle \text{component-variable} \rangle \rightarrow (\text{lbrack} + \langle \text{index-list} \rangle + \text{rbrack}) \mid (\text{period} + \text{ident})$
 $\langle \text{index-list} \rangle \rightarrow (\text{intcon} \mid \text{charcon} \mid \text{ident}) + (\text{comma} + \langle \text{index-list} \rangle)^*$

Control Flow

$\langle \text{if-statement} \rangle \rightarrow \text{ifsy} + \langle \text{expression} \rangle + \text{thensy} + \langle \text{statement} \rangle + (\text{elsy} + \langle \text{statement} \rangle)?$

$\langle \text{case-statement} \rangle \rightarrow \text{casesy} + \langle \text{expression} \rangle + \text{ofsy} + \langle \text{case-block} \rangle + \text{endsy}$
 $\langle \text{case-block} \rangle \rightarrow \langle \text{constant} \rangle + (\text{comma} + \langle \text{constant} \rangle)^* + \text{colon} + \langle \text{statement} \rangle + (\text{semicolon} + \langle \text{case-block} \rangle)^*$

$\langle \text{while-statement} \rangle \rightarrow \text{whilesy} + \langle \text{expression} \rangle + \text{dosy} + \langle \text{statement} \rangle$
 $\langle \text{repeat-statement} \rangle \rightarrow \text{repeatsy} + \langle \text{statement-list} \rangle + \text{untilsy} + \langle \text{expression} \rangle$
 $\langle \text{for-statement} \rangle \rightarrow \text{forsy} + \text{ident} + \text{becomes} + \langle \text{expression} \rangle + (\text{tosy} \mid \text{downtosy}) + \langle \text{expression} \rangle + \text{dosy} + \langle \text{statement} \rangle$

Procedure/Function Call

$\langle \text{procedure/function-call} \rangle \rightarrow \text{ident} + \text{lparent} + \langle \text{parameter-list} \rangle? + \text{rparent}$
 $\langle \text{parameter-list} \rangle \rightarrow \langle \text{expression} \rangle + (\text{comma} + \langle \text{expression} \rangle)^*$

Expression

$\langle \text{expression} \rangle \rightarrow \langle \text{simple-expression} \rangle + (\text{relational-operator} +$


```

<simple-expression>)?

<simple-expression> → (plus | minus)? + <term> + (<additive-operator> +
<term>)*

<term> → <factor> + (<multiplicative-operator> + <factor>)*
<factor> → ident | intcon | realcon | charcon | string | (lparent + <expression> +
rparent) | (notsy + <factor>) | <procedure/function-call> | <variable>

```

Operator

```

<relational-operator> → eql | neq | gtr | geq | lss | leq
<additive-operator> → plus | minus | orsy
<multiplicative-operator> → times | rdiv | idiv | imod | andsy

```

1.4 Strategi Parsing

➤ Algoritma Recursive Descent

Parser diimplementasikan menggunakan algoritma *Recursive Descent*, yaitu metode parsing top-down di mana setiap non-terminal dalam grammar direpresentasikan sebagai satu fungsi rekursif. Parsing dimulai dari start symbol <program> dan secara rekursif memanggil fungsi-fungsi turunannya hingga seluruh token habis dikonsumsi.

Keunggulan Recursive Descent:

- Grammar bersifat LL(k) sehingga setiap non-terminal dapat ditentukan cukup dengan melihat beberapa token ke depan tanpa perlu *backtracking*.
- Struktur kode mengikuti struktur grammar secara langsung, sehingga mudah dibaca dan di-*debug*.
- Mudah diintegrasikan dengan mekanisme *error handling* yang informatif.

➤ Lookahead

Beberapa situasi memerlukan lookahead (*mengintip token berikutnya*) untuk menentukan production rule yang tepat tanpa melakukan backtracking. Contoh kasusnya seperti berikut:

Situasi	Token Awal	Lookahead	Keputusan
Assignment vs procedure/function-call di statement	ident	token ke-2: becomes atau lparent	becomes → <assignment-statement> lparent → <procedure/function-

			call>
Range vs ident di array/type	ident, intcon, dan token lainnya	token ke-2 dan ke-3: period period	ada .. → <range>, tidak ada → ident
Procedure/function-call vs variable di factor	ident	token ke-2: lparent	ada lparent → <procedure/function-call>, tidak ada → <variable>

➤ **Urutan Deklarasi yang Ketat**

Berdasarkan spesifikasi, urutan deklarasi dalam <declaration-part> bersifat wajib dan tidak boleh ditukar:

const → type → var → subprogram

Parser mengimplementasikan ini secara sekuensial, bukan dengan loop bebas, sehingga apabila urutan tidak sesuai maka parser langsung menghasilkan error. Contohnya, jika var muncul sebelum type, *token* varsy tidak akan dikenali pada tahap pembacaan type-declaration dan parser akan melaporkan syntax error.

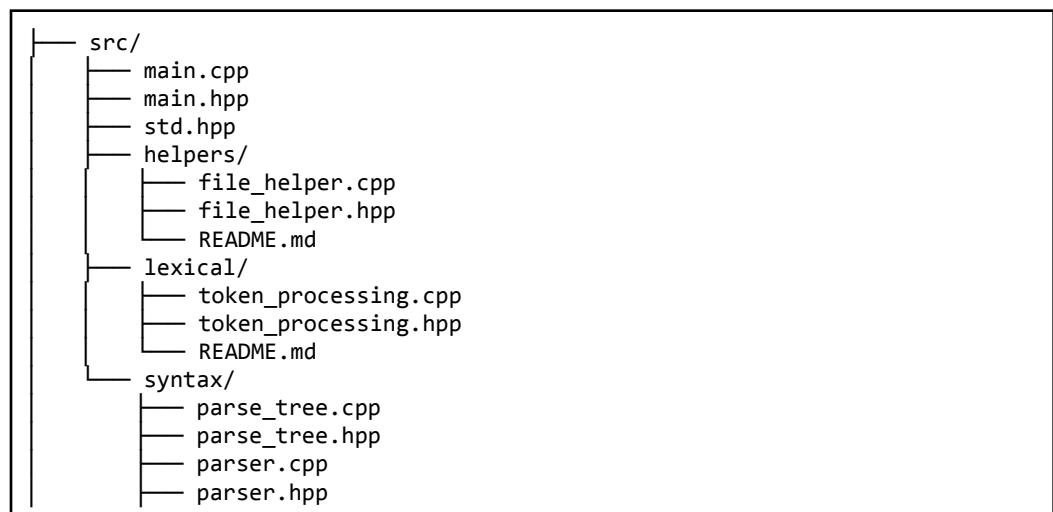
2. Implementasi

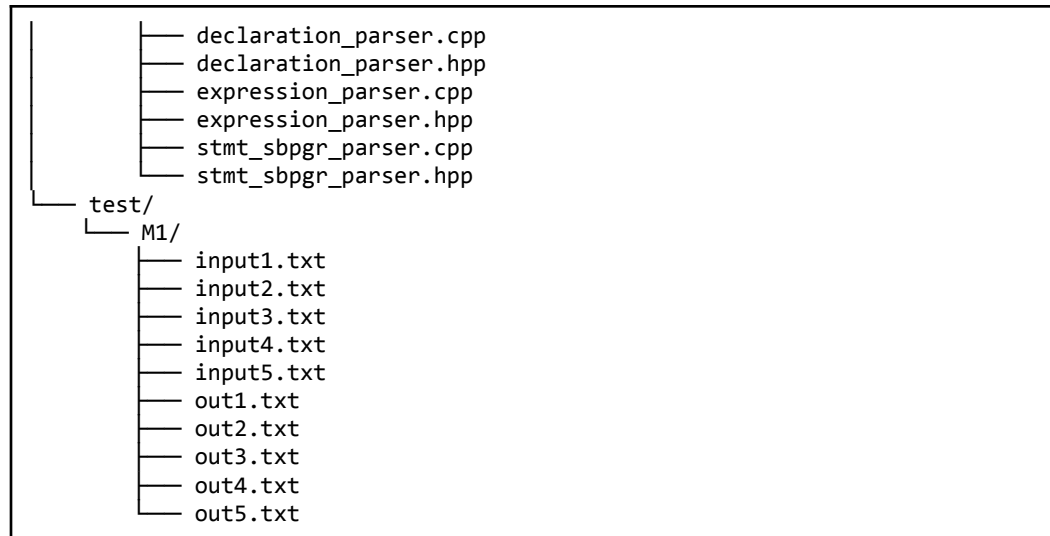
2.1 Arsitektur File

Program dibagi ke dalam beberapa modul dengan tanggung jawab yang terpisah.

File	Peran
src/std.hpp	Aggregator STL (string, vector, map, dll.)
src/main.hpp	Aggregator header
src/main.cpp	Entry point: menerima nama file, memanggil DFAFileReader → Tokenizing → TokenPrinter
src/lexical/token_processing.hpp	Mendefinisikan struct Token dan mendeklarasikan fungsi Tokenizing()
src/lexical/token_processing.cpp	Implementasi utama DFA dengan 15 state untuk tokenisasi input
src/helpers/file_helper.cpp	Fungsi pendukung: membaca file, mencetak token, menampilkan pesan error

src/syntax/parse_tree.hpp	Mendefinisikan struct ParseNode, alias NodePtr, dan mendeklarasikan fungsi makeNode(), addChild(), printTree(), saveTreeToFile()
src/syntax/parse_tree.cpp	Implementasi fungsi <i>utility</i> parse tree:
src/syntax/parser.hpp	Mendefinisikan struct ParseError dan kelas Parser sebagai entry point dan proses utama, mendeklarasikan method (peek, check, accept, expect, advance) dan mekanisme error handling
src/syntax/parser.cpp	Implementasi parseProgram() sebagai entry point dan fungsi lainnya
src/syntax/declaration_parser.hpp	Mendeklarasikan kelas DeclarationParser dan seluruh fungsi parsing deklarasi
src/syntax/declaration_parser.cpp	Implementasi seluruh fungsi deklarasi
src/syntax/stmt_sbprgr_parser.hpp	Mendeklarasikan kelas StatementSubprogramParser beserta peer parser injection (setDeclParser, setExprParser) dan seluruh fungsi parsing statement dan subprogram
src/syntax/stmt_sbprgr_parser.cpp	Implementasi parsing statement
src/syntax/expression_parser.hpp	Mendeklarasikan kelas ExpressionParser dan seluruh fungsi parsing ekspresi (expression, simple-expression, term, factor)
src/syntax/expression_parser.cpp	Implementasi parsing ekspresi





2.2 Struktur Data

- **Token**

Token dihasilkan oleh lexer dan menjadi input parser dengan struktur:

```

struct Token{
    string type;
    string value;
};
  
```

- **ParseNode**

Node parse tree direpresentasikan sebagai:

```

struct ParseNode {
    string label;
    vector<shared_ptr<ParseNode>> children;

    explicit ParseNode(const string& lbl) : label(lbl) {}
};

using NodePtr = shared_ptr<ParseNode>;
  
```

Setiap node dapat memiliki anak sejumlah apapun sesuai produksi grammar-nya. Node terminal (*token*) tidak memiliki anak, sedangkan node non-terminal seperti <program> dan <declaration-part> memiliki anak yang merepresentasikan komponen-komponennya.

- **ParseError (*Error information*)**

```

struct ParseError {
    string message;
    int tokenIndex;
    string tokenType;
    string tokenValue;
};
  
```

Terdapat 2 mekanisme *error handling* yang digunakan:

1) Error Reporting

Setiap kali *parser* menemukan *token* yang tidak sesuai ekspektasi, *reportError()* dipanggil untuk mencatat error ke dalam daftar errors. Pesan error bersifat informatif sehingga memudahkan *user* mengidentifikasi letak kesalahan.

2) Recovery Mode

Setelah error dicatat, *synchronize()* dipanggil untuk melanjutkan *parsing* dengan cara melompat ke *token* sinkronisasi terdekat seperti semicolon, beginsy, atau endsy. Mekanisme ini memungkinkan *parser* mendeteksi lebih dari satu error dalam satu kali jalan tanpa berhenti di error pertama sehingga *user* mendapatkan laporan error yang lebih lengkap sekaligus.

→ Terdapat fungsi *utility* yang dapat digunakan untuk memanipulasi tree

Fungsi	Kegunaan
makeNode(label)	Membuat node baru dengan label tertentu
addChild(parent, child)	Menambahkan node anak ke node parent
printTree(root)	Mencetak tree ke terminal
saveTreeToFile(root, path)	Menyimpan tree ke file .txt

2.3 Implementasi Parser

Parser

Kelas Parser berperan sebagai proses utama yang menyimpan referensi ke seluruh list token dan menyediakan fungsi/*method* yang digunakan bersama oleh semua sub-parser:

Fungsi	Kegunaan
peek(offset)	Melihat token ke depan tanpa mengonsumsi
check(type)	Mengecek tipe token saat ini
accept(type)	Mengonsumsi token jika tipenya cocok

expect(type, ctx)	Mengonsumsi token, error jika tidak cocok
advance()	Memajukan posisi parser satu token
reportError(message)	Mencatat error ke daftar errors

Parsing dimulai dari *parseProgram()* yang mengikuti produksi:

<program> → <program-header> + <declaration-part> +
<compound-statement> + period

Parser.hpp

```
class Parser {
public:
    Parser(const vector<Token>& toks);

    //entry point: parse <program>
    NodePtr parseProgram();

    bool hasError() const { return !errors.empty(); }
    const vector<ParseError>& getErrors() const { return errors; }

    //---- core engine ----
    //token consumption + lookahead, dipakai sama semua sub-parser
    const Token& peek(int offset = 0) const;
    bool check(const string& type) const;
    bool accept(const string& type); //consume kalau cocok
    bool expect(const string& type, const string& ctx); //error kalau ga
    cocok
    const Token& advance();
    bool isAtEnd() const;

    void reportError(const string& message);

    //---- grammar rules ----
    NodePtr parseProgramHeader();
    NodePtr parseDeclarationPart();
    NodePtr parseCompoundStatement();
    NodePtr parseStatementList();

    //---- stub ----
    NodePtr parseDeclaration();
    NodePtr parseStatement();
}
```

```
private:
    const vector<Token>& tokens;
    int pos;
    vector<ParseError> errors;

    DeclarationParser declParser;
    ExpressionParser exprParser;
    StatementSubprogramParser stmtParser;

    void skipTrivia(); //auto-skip token komentar
    void synchronize(const vector<string>& syncTypes); //panic-mode recovery

    static string tokenDisplay(const Token& t);
};
```

DeclarationParser

Menangani seluruh bagian deklarasi program. Fungsi utamanya adalah *parseDeclarationPart()* yang memproses deklarasi secara sekuensial sesuai urutan *const* → *type* → *var* → *subprogram*, khusus *subprogram* di-handle *StatementSubprogramParser*. Fungsi-fungsi turunannya:

Fungsi	Kegunaan
<code>parseDeclarationPart()</code>	Memproses seluruh declaration part secara sekuensial
<code>parseConstDeclaration()</code>	Memproses blok const
<code>parseTypeDeclaration()</code>	Memproses blok type termasuk array, range, enumerated, record
<code>parseVarDeclaration()</code>	Memproses blok var
<code>parseConstant()</code>	Memproses nilai konstanta dan memisahkan kasus charcon/string dari yang boleh diberi tanda +/-
<code>parseIdentifierList()</code>	Memproses daftar identifier yang dipisahkan koma
<code>parseArrayType()</code>	Memproses tipe array
<code>parseType()</code>	Memproses tipe data (ident, array, range, enumerated, record)
<code>parseRange()</code>	Memproses range constant..constant

parseEnumerated()	Memproses tipe enumerated
parseRecordType()	Memproses tipe record beserta field-list-nya
parseFieldList()	Memproses daftar field dengan penanganan semicolon berlebih sebelum end
parseFieldPart()	Memproses satu field identifier-list : type

declaration_parser.hpp

```

class DeclarationParser {
public:
    // New parser dari list token hasil lexer
    explicit DeclarationParser(const vector<Token>& tokens);

    // Parsing deklarasi
    NodePtr parseDeclarationPart();

    NodePtr parseConstDeclaration();
    NodePtr parseTypeDeclaration();
    NodePtr parseVarDeclaration();
    NodePtr parseConstant();
    NodePtr parseIdentifierList();
    NodePtr parseArrayType();

    void setPosition(size_t p);
    size_t getPosition() const;

private:
    const vector<Token>& tokens;
    size_t pos;

    // Cek apakah token bisa mulai constant (curr pos)
    bool isConstantStart(size_t offset = 0) const;

    // Cek apakah token membentuk range (curr pos)
    bool isRangeAhead(size_t offset = 0) const;

    NodePtr parseProgramHeader(); // program header
    NodePtr parseType(); // type
    NodePtr parseRange(); // range constant..constant
    NodePtr parseEnumerated(); // enumerated
    NodePtr parseRecordType(); // record type

```



```

NodePtr parseFieldList(); // field list
NodePtr parseFieldPart(); // field part
};

```

StatementSubprogramParser

Menangani parsing statement dan deklarasi subprogram. Kelas ini menggunakan referensi ke DeclarationParser dan ExpressionParser yang disisipkan dari luar sehingga kelas ini dapat langsung menyerahkan tugas tersebut ke kelas yang tepat.

Fungsi parseStatement() melakukan lookahead untuk menentukan jenis statement:

Kondisi	Fungsi yang dipanggil
ident + becomes	parseAssignmentStatement()
Ident + lparent	parseProcedureFunctionCall()
ifsy	parseIfStatement()
casesy	parseCaseStatement()
whilesy	parseWhileStatement()
repeatsy	parseRepeatStatement()
forsy	parseForStatement()
Tidak ada yang cocok	Node <statement> kosong (ϵ)

parseCaseBlock()	Memproses satu blok case beserta konstanta, colon, dan statement-nya, dipanggil secara rekursif dari parseCaseStatement()
parseStatementList()	Memproses daftar statement yang dipisahkan semicolon, dipanggil dari parseCompoundStatement() dan parseRepeatStatement()

Variable & Component

Fungsi	Kegunaan
--------	----------

parseVariable()	Memproses ident + (component-variable)*
parseComponentVariable()	Memproses akses array [index-list] atau field record .ident
parseIndexList()	Memproses daftar index dalam akses array

Subprogram

Fungsi	Kegunaan
parseSubprogramDeclaration()	Membedakan procedure dan function
parseProcedureDeclaration()	Memproses deklarasi procedure
parseFunctionDeclaration()	Memproses deklarasi function
parseBlock()	Memproses block yang secara rekursif memanggil parseDeclarationPart() dan parseCompoundStatement()
parseFormalParameterList()	Memproses daftar parameter formal
parseParameterGroup()	Memproses satu kelompok parameter identifier-list : type
parseParameterList()	Memproses daftar parameter aktual saat pemanggilan

stmt_sbpgpr_parser.hpp

```
class StatementSubprogramParser {
public:
    // konstruktor
    explicit StatementSubprogramParser() = default;
    explicit StatementSubprogramParser(const vector<Token> &tokens);

    // setter dan getter
    void setTokens(const vector<Token> &tokens);
    void setPosition(size_t pos);
    size_t getPosition() const;

    // peer parser injection (dipakai utk delegasi internal stub)
    void setDeclParser(DeclarationParser* p);
    void setExprParser(ExpressionParser* p);
};
```

```

// parser untuk statements
NodePtr parseStatement();
NodePtr parseAssignmentStatement();
NodePtr parseIfStatement();
NodePtr parseCaseStatement();
NodePtr parseWhileStatement();
NodePtr parseRepeatStatement();
NodePtr parseForStatement();
NodePtr parseProcedureFunctionCall();

// parser untuk parameter
NodePtr parseParameterList();
NodePtr parseVariable();
NodePtr parseComponentVariable();
NodePtr parseIndexList();

// parse untuk deklarasi program
NodePtr parseSubprogramDeclaration();
NodePtr parseProcedureDeclaration();
NodePtr parseFunctionDeclaration();
NodePtr parseBlock();
NodePtr parseFormalParameterList();
NodePtr parseParameterGroup();

// method untuk parse eksternal
NodePtr parseCaseBlock();
NodePtr parseStatementList();

private:
// atribut
vector<Token> tokens;
size_t pos = 0;

// peer parsers (non-owning)
DeclarationParser* declParser_ = nullptr;
ExpressionParser* exprParser_ = nullptr;
};

```

ExpressionParser

Menangani parsing ekspresi dengan mengikuti hirarki prioritas operator dari rendah ke tinggi:

<expression>	— relational (=, <>, <, >, <=, >=)
└─ <simple-expression>	— additive (+, -, or)
└─ <term>	— multiplicative (*, /, div, mod, and)
└─ <factor>	— atom (literal, ident, kurung, not)

Fungsi	Kegunaan
parseExpression()	Memproses <i>relational</i> operator antar simple-expression
parseSimpleExpression()	Memproses <i>additive</i> operator antar term, termasuk tanda +/- di awal
parseTerm()	Memproses <i>multiplicative</i> operator antar factor
parseFactor()	Memproses unit terkecil ekspresi: literal, kurung, not, variable, atau procedure/function-call
parseVariable()	Memproses ident + (component-variable)*, dipanggil dari parseFactor()
parseProcedureFunctionCall()	Memproses pemanggilan procedure/function, dipanggil dari parseFactor()

Khusus parseFactor(), lookahead digunakan untuk membedakan dua kasus yang sama-sama diawali ident:

- ident + lparent → parseProcedureFunctionCall()
- ident tanpa lparent → parseVariable()

expression_parser.hpp

```
class ExpressionParser {
public:
    // konstruktor
    explicit ExpressionParser() = default;
    explicit ExpressionParser(const vector<Token> &tokens);

    // setter dan getter
    void setTokens(const vector<Token> &tokens);
    void setPosition(size_t pos);
    size_t getPosition() const;
```

```

// parser untuk statements
NodePtr parseExpression();
NodePtr parseSimpleExpression();
NodePtr parseTerm();
NodePtr parseFactor();

private:
NodePtr parseVariable();
NodePtr parseProcedureFunctionCall();

// atribut
vector<Token> tokens;
size_t pos = 0;

// helper untuk mengecek token
Token currentToken() const;
Token peek(int offset = 1) const;
bool isAtEnd() const;
void advance();
bool matchType(const string &type);
bool matchValue(const string &value);
void consume(const string &expectedType, const string
              &errorMessage);
};

```

BAB III

PENGUJIAN

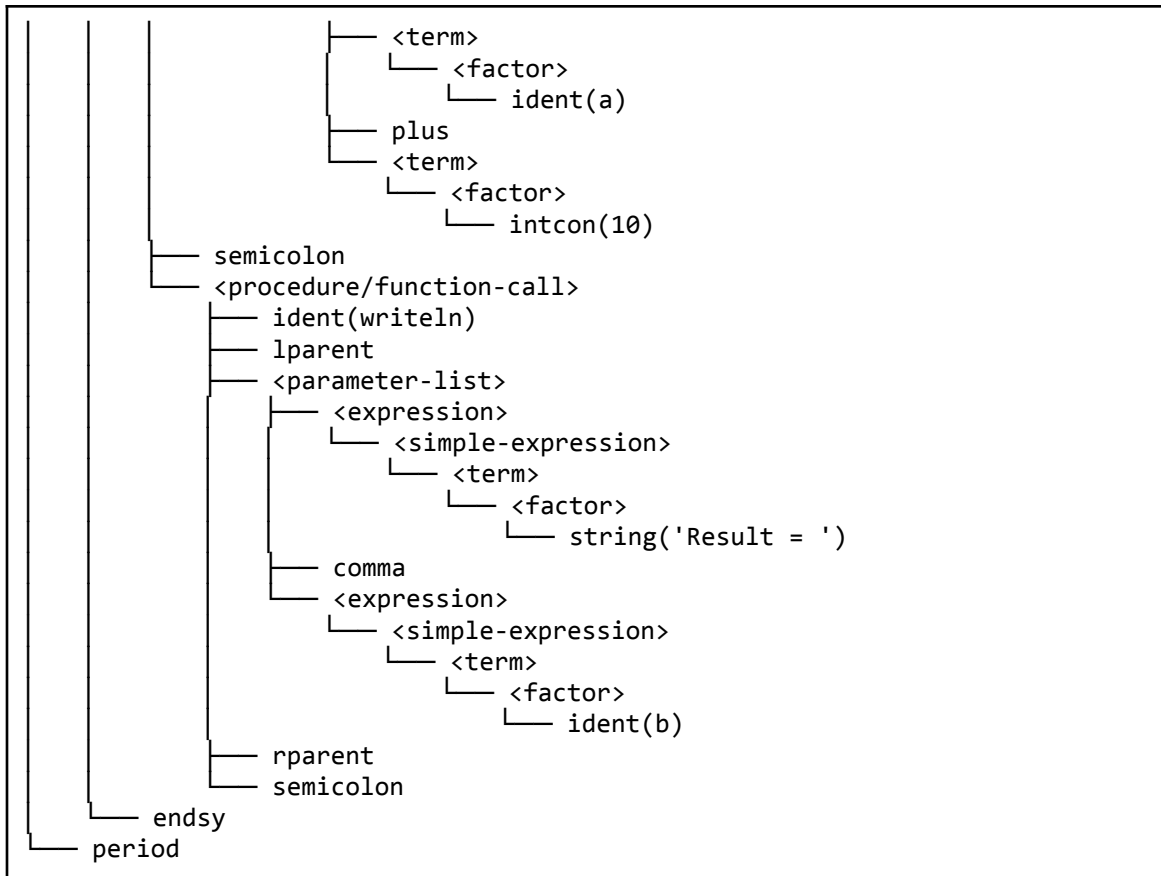
1. Test Case 1

Input :

```
program Hello;  
  
var  
  a, b: integer;  
  
begin  
  a := 5;  
  b := a + 10;  
  writeln('Result = ', b);  
end.
```

Output :

```
<program>  
├── <program-header>  
│   ├── programsy  
│   ├── ident(Hello)  
│   └── semicolon  
├── <declaration-part>  
│   └── <var-declaration>  
│       ├── varsy  
│       └── <var-item>  
│           ├── <identifier-list>  
│           │   ├── ident(a)  
│           │   ├── comma  
│           │   └── ident(b)  
│           ├── colon  
│           ├── <type>  
│           │   └── ident(integer)  
│           └── semicolon  
└── <compound-statement>  
    ├── beginsy  
    └── <statement-list>  
        ├── <assignment-statement>  
        │   ├── ident(a)  
        │   ├── becomes  
        │   └── <expression>  
        │       └── <simple-expression>  
        │           └── <term>  
        │               └── <factor>  
        │                   └── intcon(5)  
        ├── semicolon  
        └── <assignment-statement>  
            ├── ident(b)  
            ├── becomes  
            └── <expression>  
                └── <simple-expression>
```



2. Test Case 2

Input :

```

program AllDecl;

const
  X == 10;
  PI == 3.14;

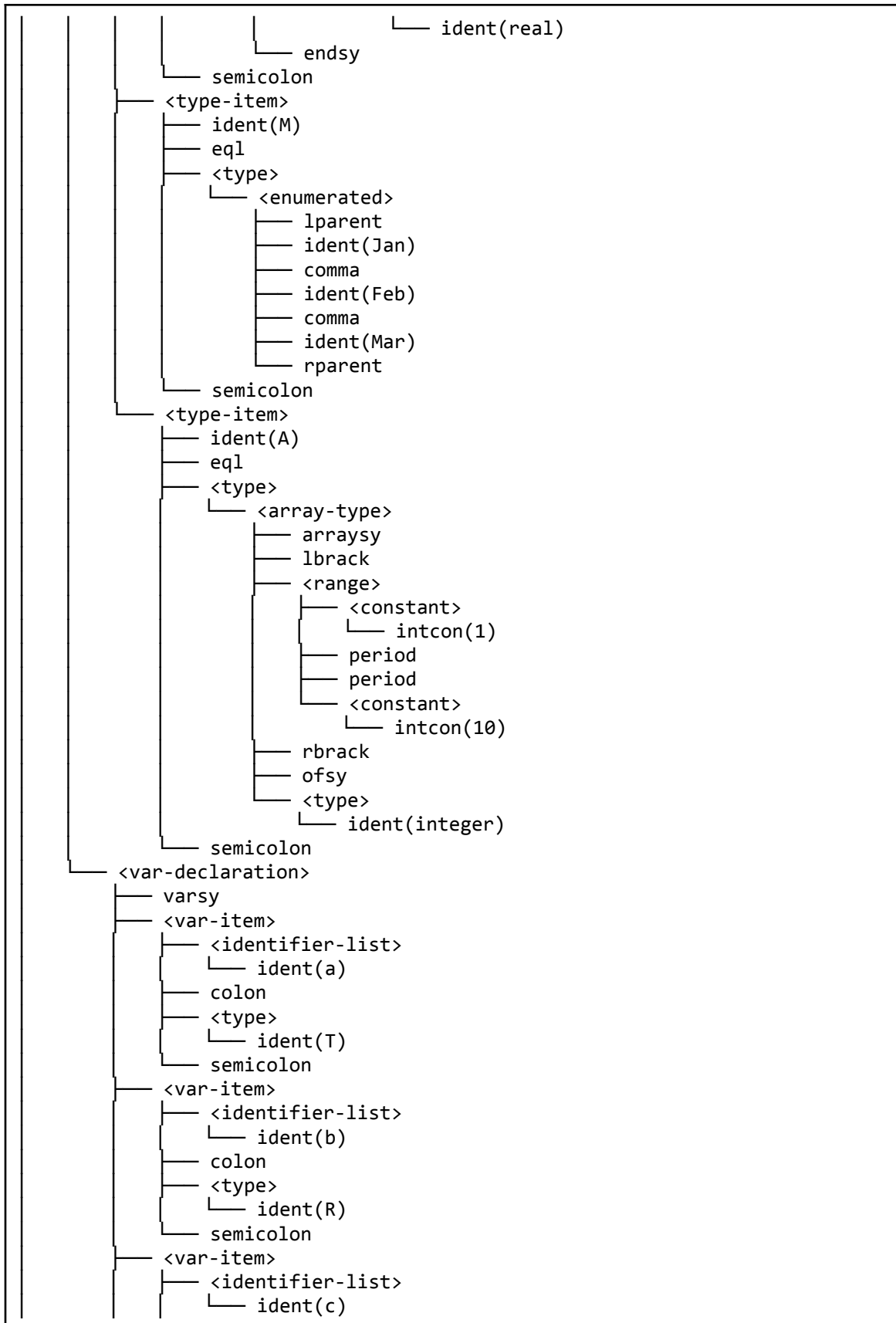
type
  T == integer;
  R == record
    a: integer;
    b: real;
  end;
  M == (Jan, Feb, Mar);
  A == array[1..10] of integer;

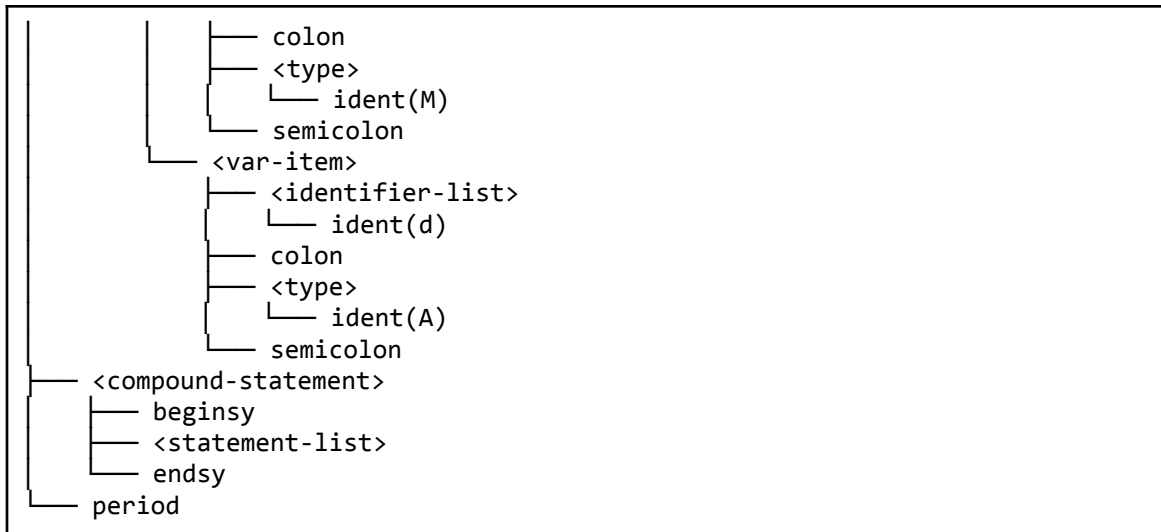
var
  a: T;
  b: R;
  c: M;
  d: A;
  
```

```
begin
end.
```

Output :

```
<program>
├── <program-header>
│   ├── programsy
│   ├── ident(AllDecl)
│   └── semicolon
├── <declaration-part>
│   ├── <const-declaration>
│   │   ├── constsy
│   │   ├── <const-item>
│   │   │   ├── ident(X)
│   │   │   ├── eql
│   │   │   └── <constant>
│   │   │       └── intcon(10)
│   │   └── semicolon
│   ├── <const-item>
│   │   ├── ident(PI)
│   │   ├── eql
│   │   └── <constant>
│   │       └── realcon(3.14)
│   └── semicolon
├── <type-declaration>
│   ├── typesy
│   ├── <type-item>
│   │   ├── ident(T)
│   │   ├── eql
│   │   └── <type>
│   │       └── ident(integer)
│   └── semicolon
├── <type-item>
│   ├── ident(R)
│   ├── eql
│   └── <type>
│       └── <record-type>
│           ├── recordsy
│           └── <field-list>
│               ├── <field-part>
│               │   ├── <identifier-list>
│               │   │   └── ident(a)
│               │   ├── colon
│               │   └── <type>
│               │       └── ident(integer)
│               └── semicolon
│               ├── <field-part>
│               │   ├── <identifier-list>
│               │   │   └── ident(b)
│               │   ├── colon
│               │   └── <type>
│               └── <type>
```



3. Test Case 3

Input :

```

program Control;

var
  a, b, i: integer;

begin
  if a > 0 then
    b := 1
  else
    b := 0;

  case a of
    1: b := 1;
    2, 3: b := 2;
  end;

  while a < 10 do
    a := a + 1;

  repeat
    a := a - 1;
  until a == 0;

  for i := 1 to 10 do
    a := i;
  end.

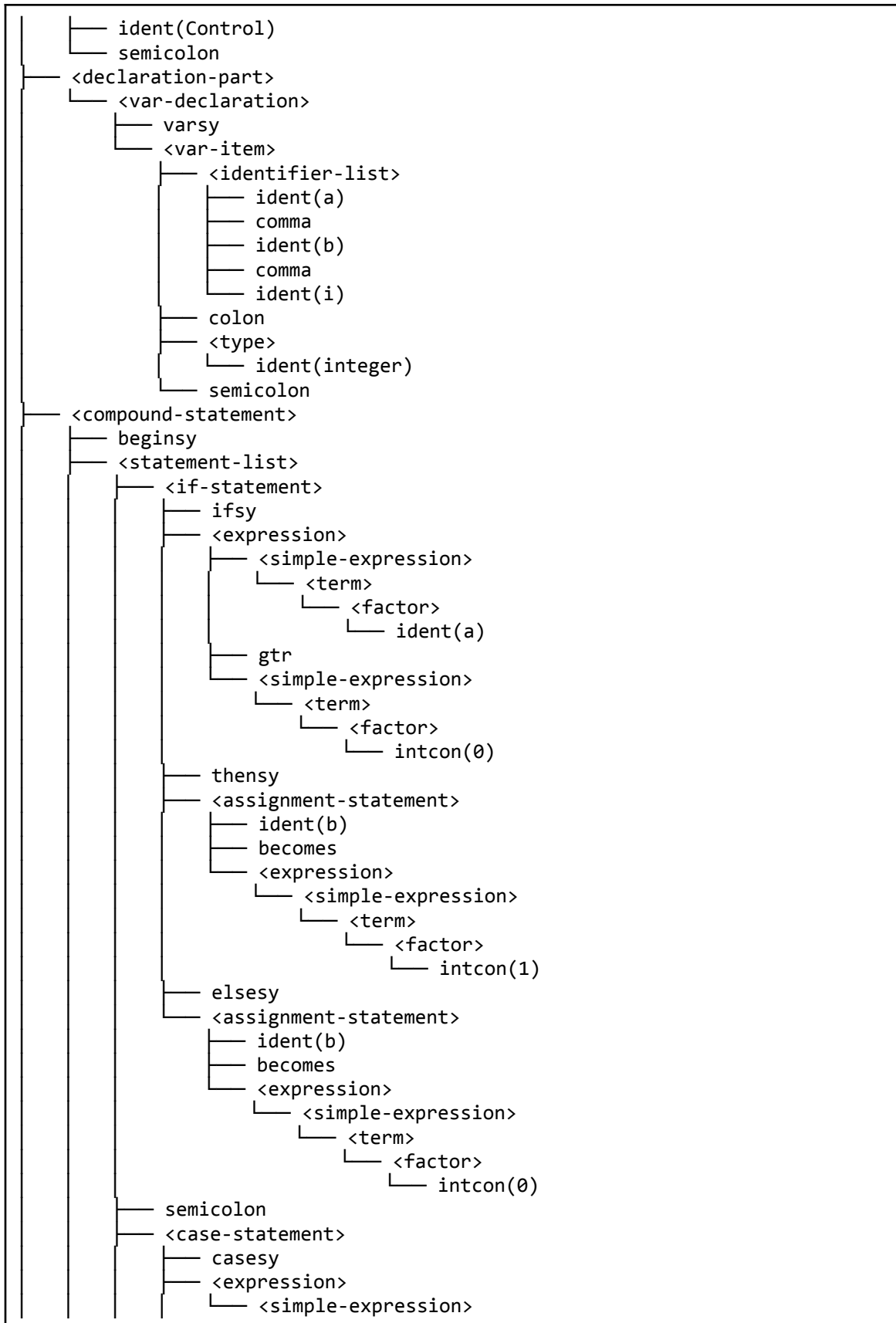
```

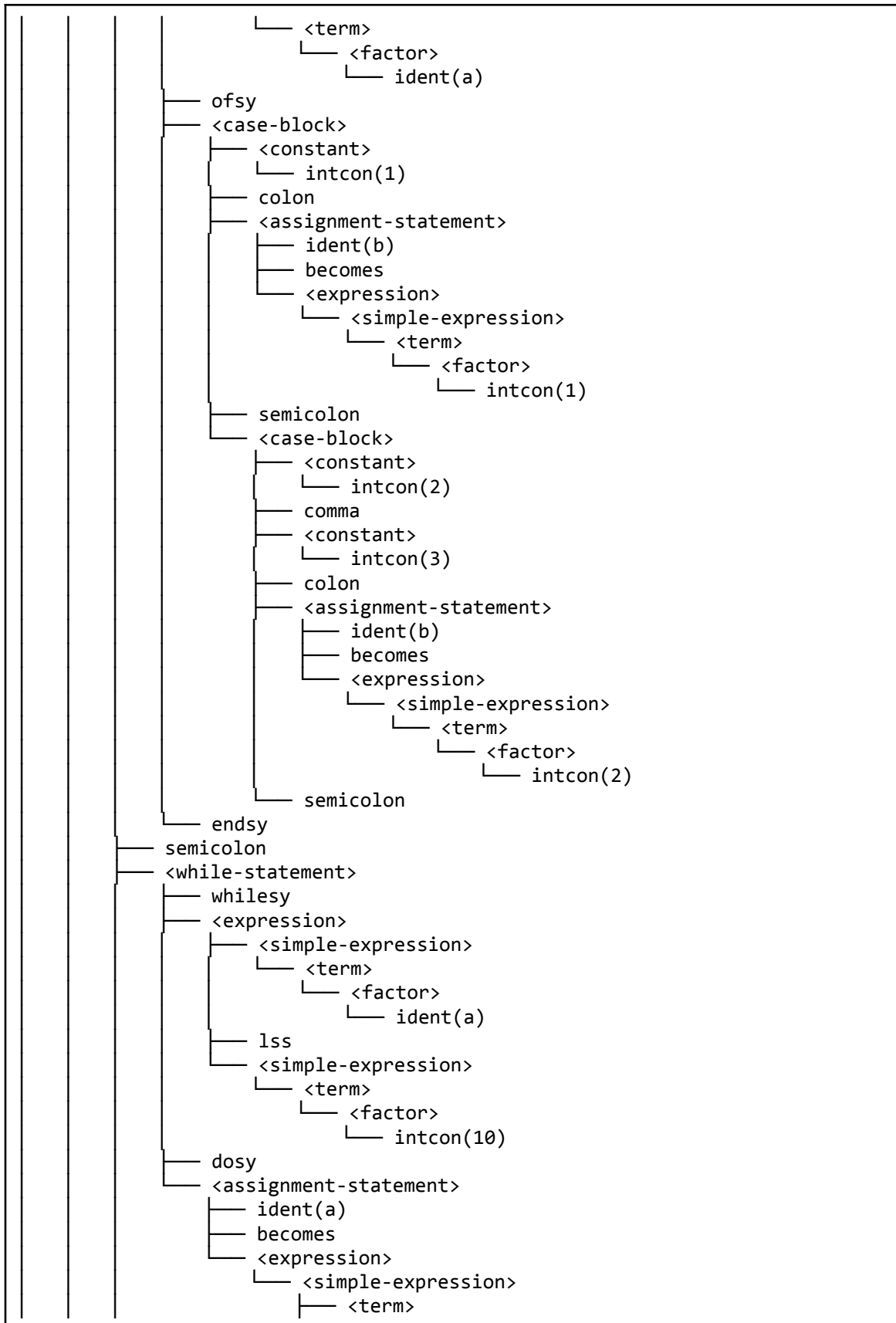
Output :

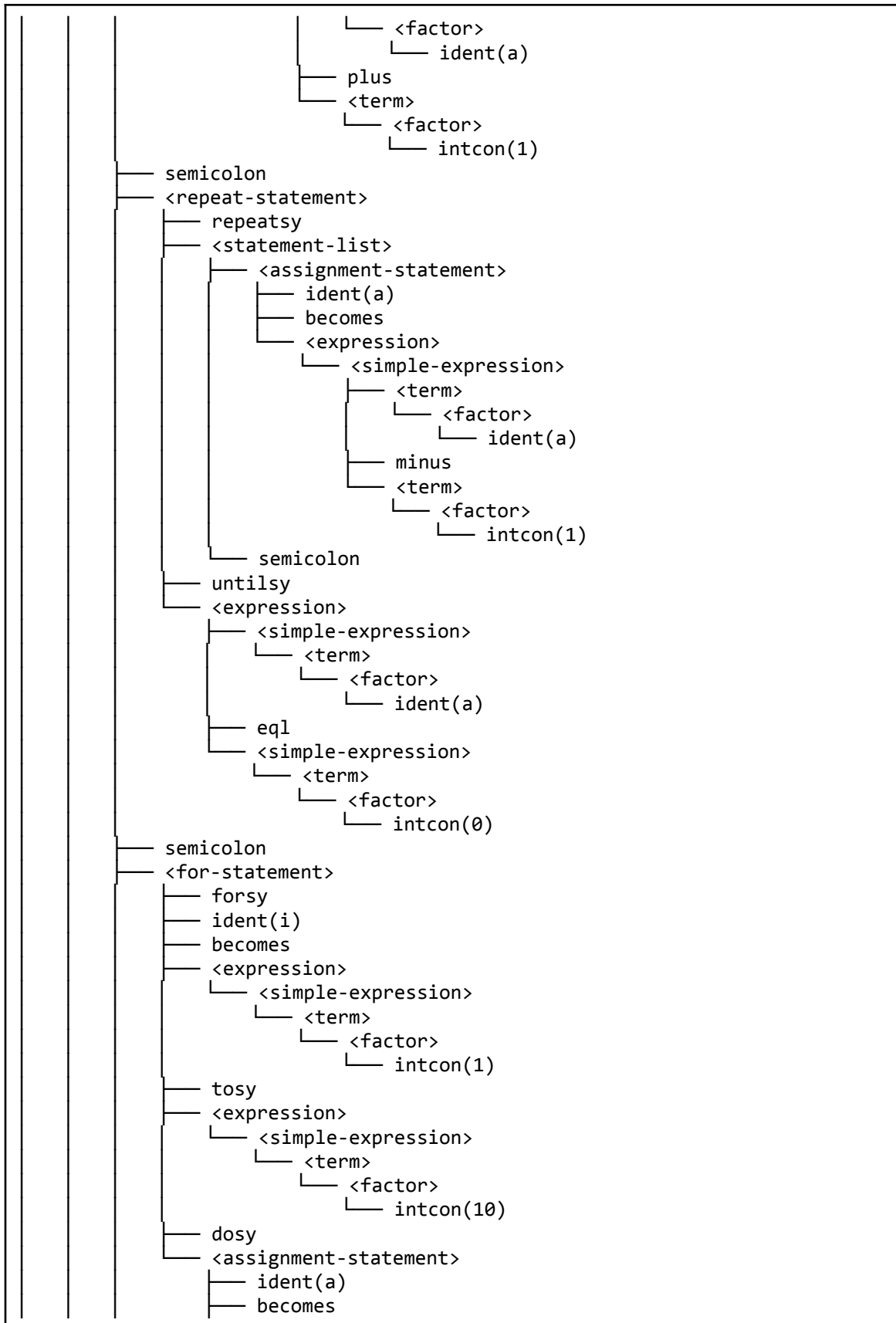
```

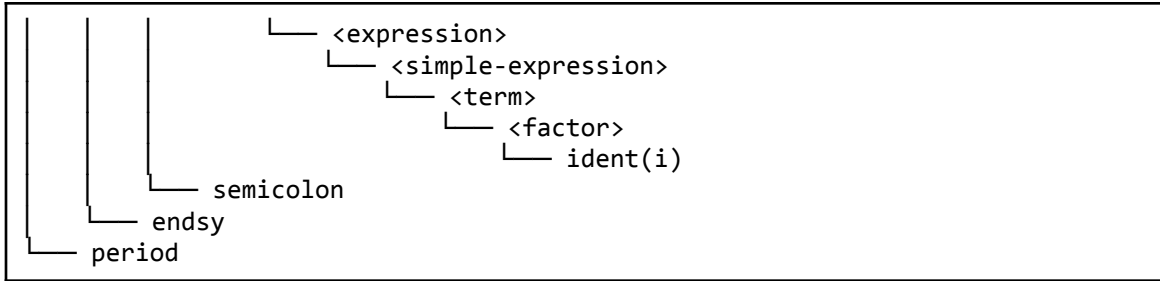
<program>
├── <program-header>
│   └── programsy

```









4. Test Case 4

Input :

```

program Subprograms;

procedure Swap(a, b: integer);
var
  temp: integer;
begin
  temp := a;
  a := b;
  b := temp;
end;

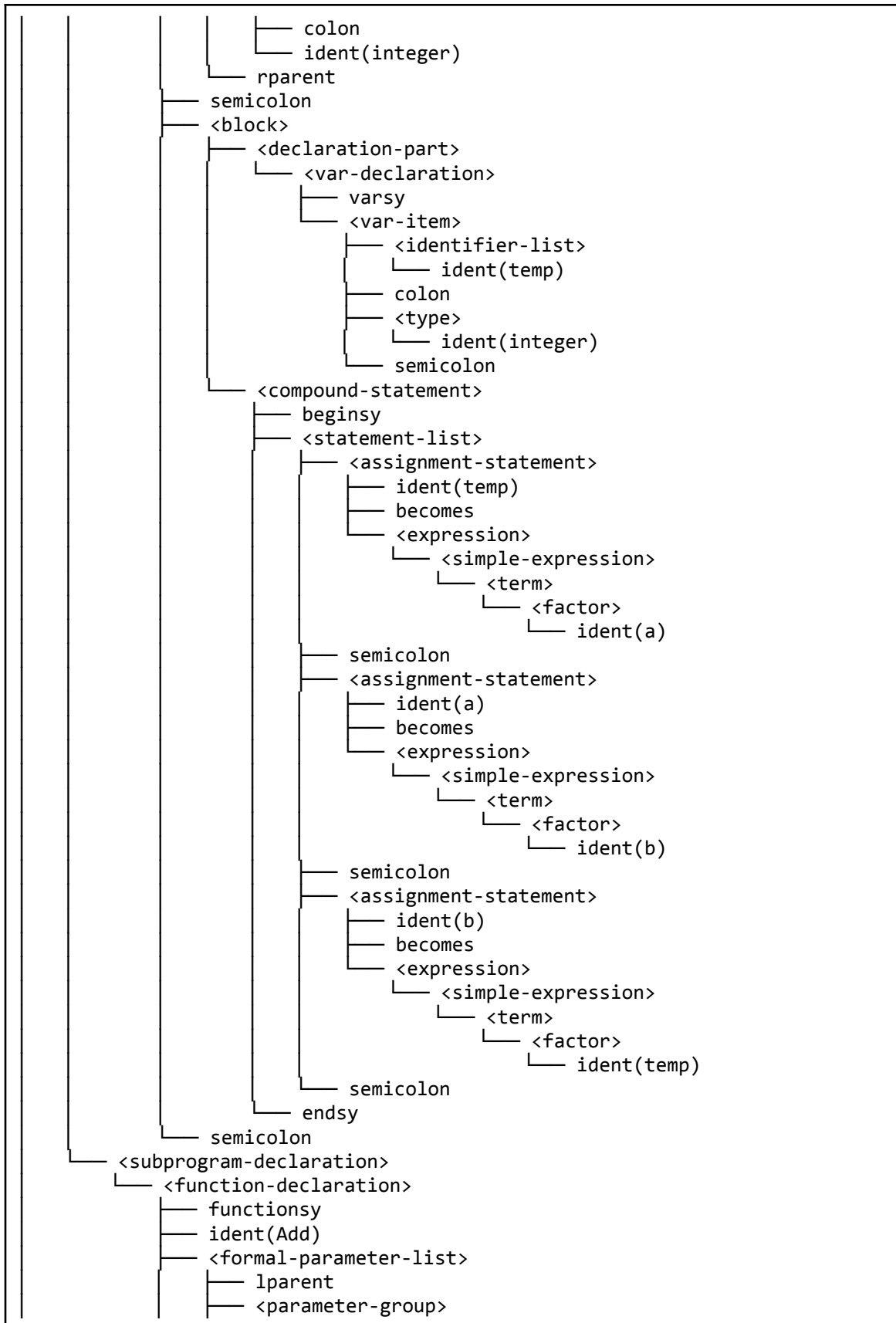
function Add(a, b: integer): integer;
begin
  Add := a + b;
end;

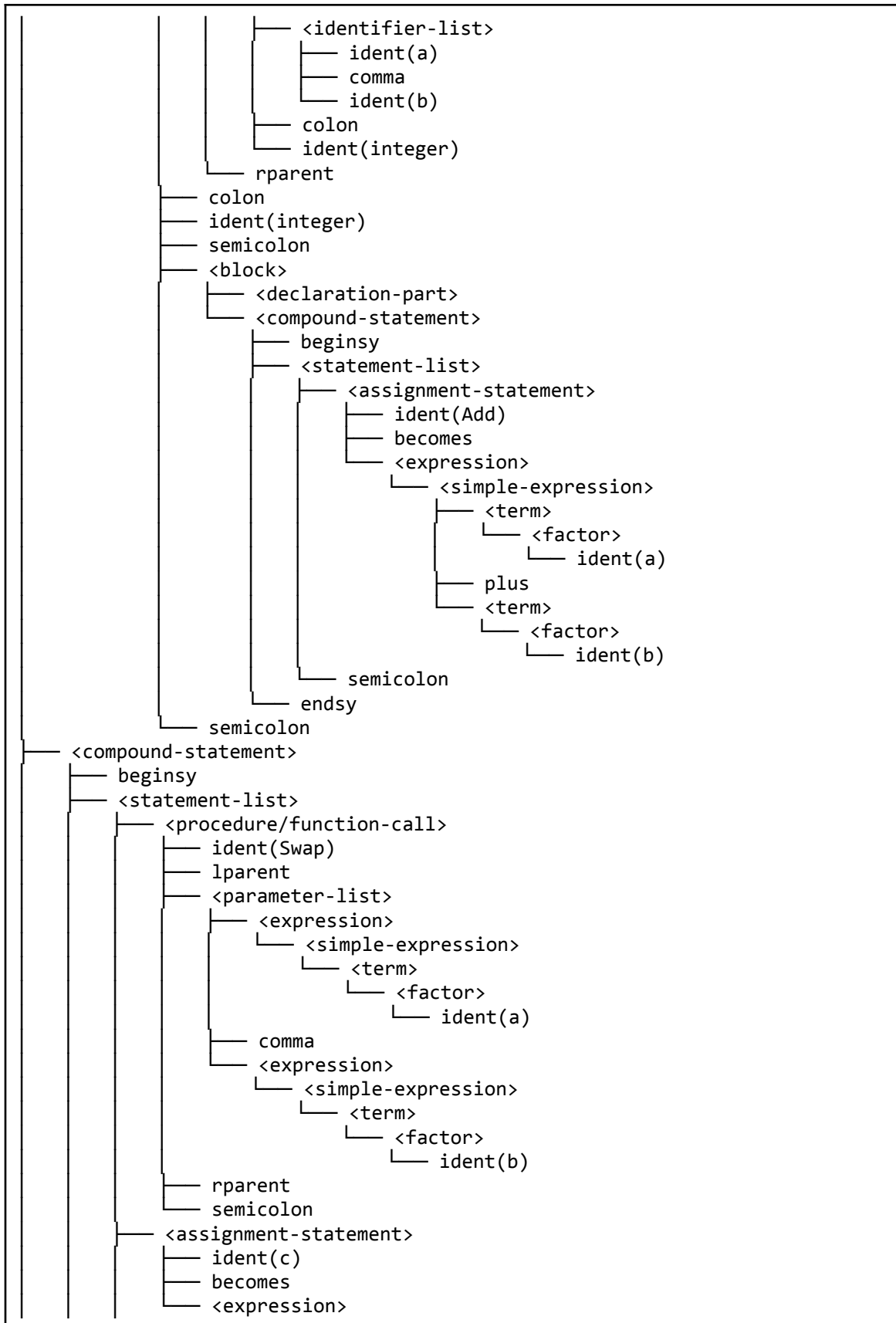
begin
  Swap(a, b);
  c := Add(1, 2);
end.
  
```

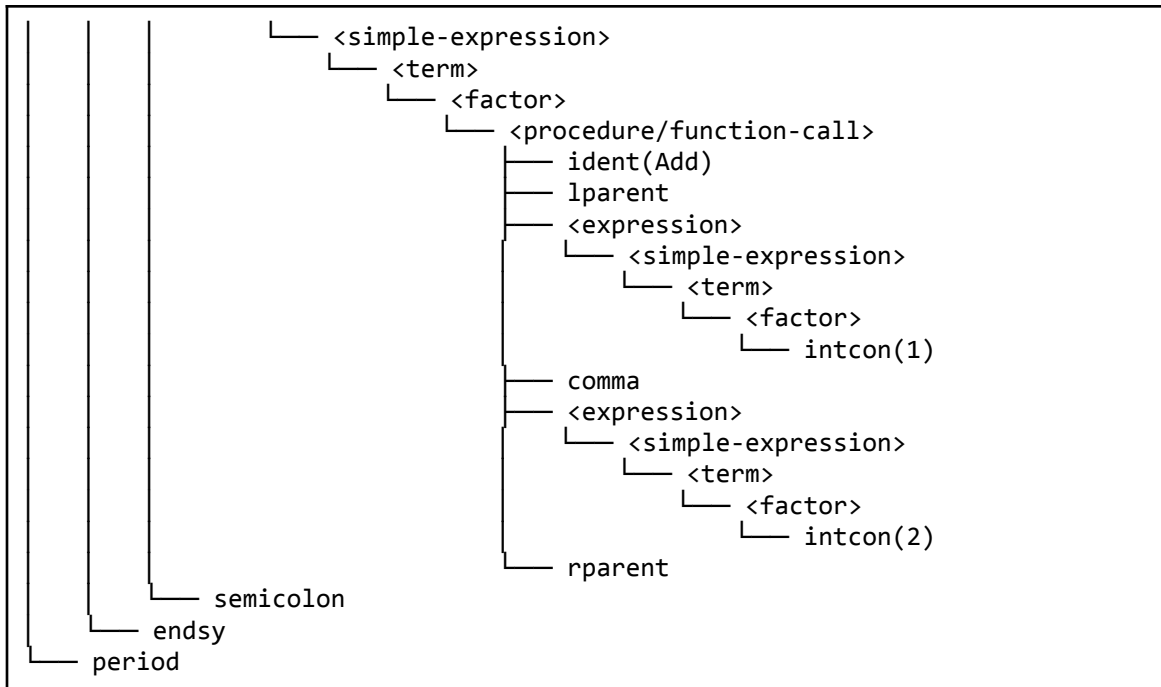
Output :

```

<program>
├── <program-header>
│   ├── programsy
│   ├── ident(Subprograms)
│   └── semicolon
├── <declaration-part>
│   ├── <subprogram-declaration>
│   │   ├── <procedure-declaration>
│   │   │   ├── proceduresy
│   │   │   ├── ident(Swap)
│   │   │   └── <formal-parameter-list>
│   │   │       ├── lparent
│   │   │       ├── <parameter-group>
│   │   │       │   ├── <identifier-list>
│   │   │       │   │   ├── ident(a)
│   │   │       │   │   ├── comma
│   │   │       │   │   └── ident(b)
│   │   │       └── rparent
│   │   └── semicolon
│   └── <function-declaration>
│       ├── functionesy
│       ├── ident(Add)
│       └── <formal-parameter-list>
│           ├── lparent
│           ├── <parameter-group>
│           │   ├── <identifier-list>
│           │   │   ├── ident(a)
│           │   │   ├── comma
│           │   │   └── ident(b)
│           └── rparent
└── <begin-end>
    ├── beginsy
    ├── Swap(a, b);
    ├── c := Add(1, 2);
    └── endsy
  
```







5. Test Case 5

Input :

```

program Expr;

var
  a, b, c, d: integer;

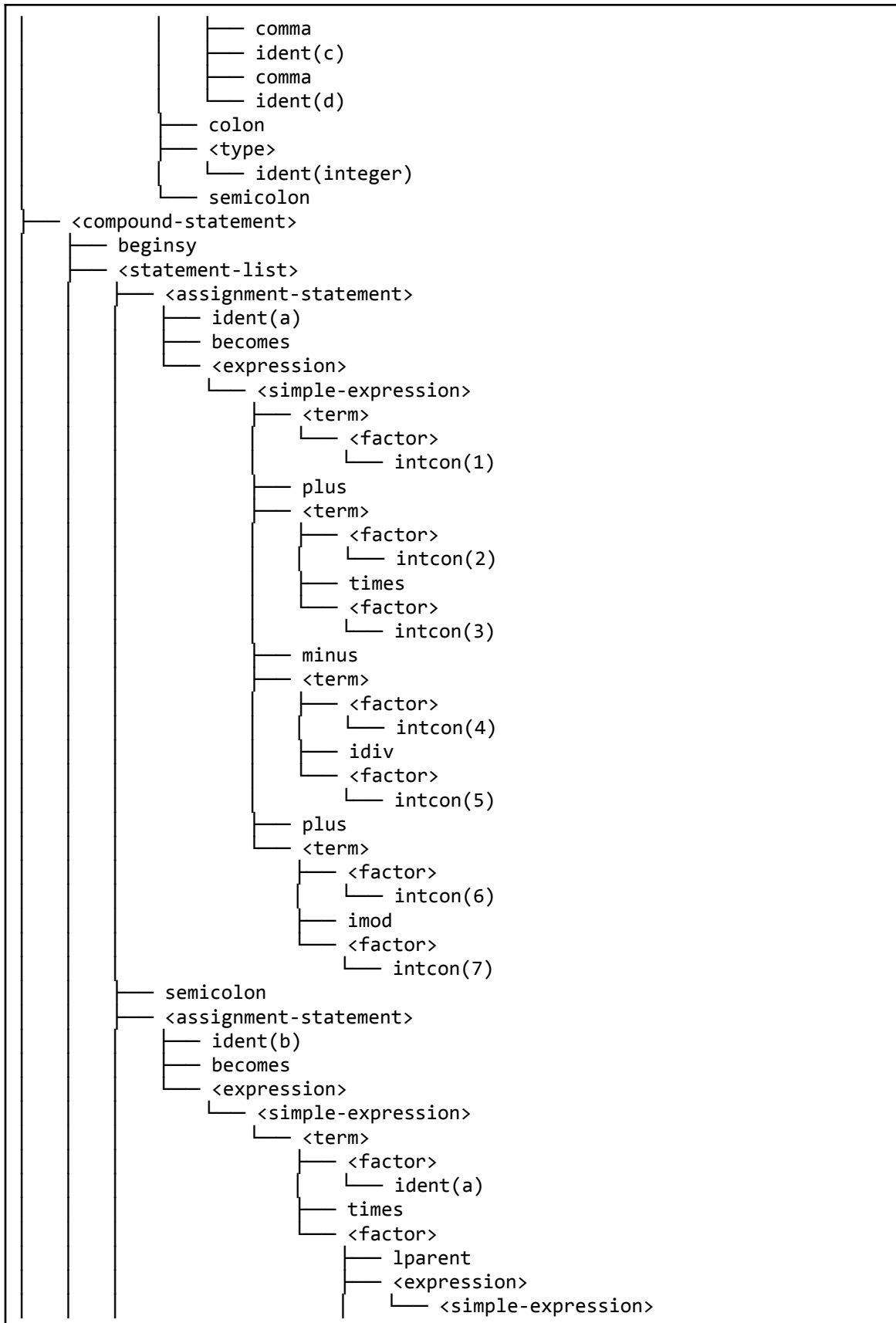
begin
  a := 1 + 2 * 3 - 4 div 5 + 6 mod 7;
  b := a * (b + c);
  c := not a and b or c;
  d := (a == b) and (c <> d);
end.

```

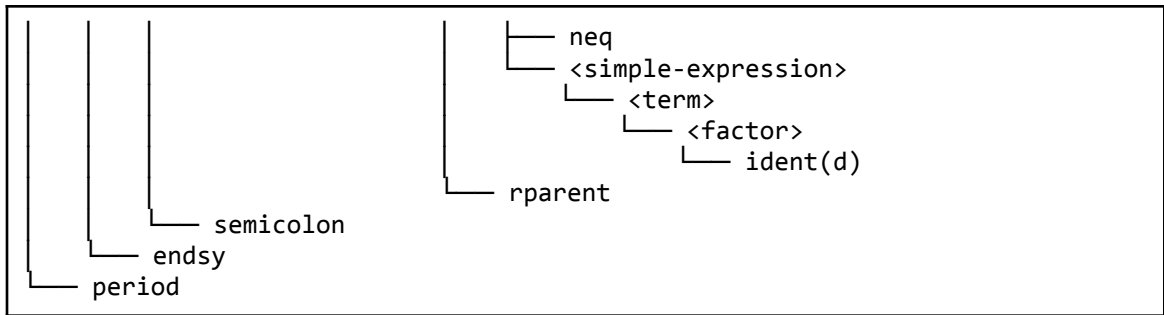
Output :

```

<program>
├── <program-header>
│   ├── programsy
│   ├── ident(Expr)
│   └── semicolon
└── <declaration-part>
    ├── <var-declaration>
    │   ├── varsy
    │   └── <var-item>
    │       ├── <identifier-list>
    │       │   ├── ident(a)
    │       │   ├── comma
    │       │   └── ident(b)
    
```







BAB IV

KESIMPULAN DAN SARAN

1. Kesimpulan

Pada milestone kedua dari tugas besar ini telah berhasil diimplementasikan sebuah Syntax Analyzer untuk compiler-interpreter dari bahasa Arion dengan metode Recursive Descent Parser. Parser menerima masukan berupa daftar token hasil proses Lexical Analysis yang sudah dibuat di milestone sebelumnya dan menghasilkan struktur hierarkis berupa Parse Tree yang merepresentasikan susunan sintaksis program sesuai aturan grammar yang telah dirancang.

Grammar yang digunakan dan disediakan, disusun dalam bentuk Context Free Grammar (CFG) dengan start symbol <program>. Grammar tersebut mencakup berbagai konstruksi bahasa seperti deklarasi konstanta, tipe, variabel, subprogram, statement kontrol, procedure/function call, serta expression dengan prioritas operator yang benar. Implementasi parser dilakukan secara modular dengan pemisahan komponen parser menjadi beberapa kelas seperti *Parser*, *DeclarationParser*, *StatementSubprogramParser*, dan *ExpressionParser* sehingga struktur program menjadi lebih terorganisiri dan mudah dikembangkan.

Selain itu, parser juga melakukan mekanisme lookahead untuk membedakan beberapa produksi yang memiliki awalan token sama tanpa perlu melakukan backtracking. Mekanisme error handling dan recovery mode juga berhasil diterapkan sehingga parser mampu melaporkan lebih dari satu kesalahan sintaks dalam satu kali proses parsing.

Berdasarkan hasil pengujian pada berbagai test case, parser berhasil menangani berbagai bentuk program yang mencakup deklarasi, statement kontrol, subprogram, dan expression kompleks sesuai dengan grammar yang telah didefinisikan. Parse tree yang dihasilkan juga menunjukkan bahwa struktur sintaks program berhasil direpresentasikan dengan benar. Dengan demikian, tujuan utama milestone ini, yaitu membangun Syntax Analyzer berbasis Recursive Descent Parsing untuk Arion Compiler, telah berhasil dicapai.

2. Saran

Beberapa saran yang bisa diberikan selama pengerjaan ini adalah:

Kepada Kelompok:

- Membagi implementasi parser agar lebih jelas sehingga mengurangi beban ketika melakukan integrasi
- Melakukan perencanaan matang yang lebih runut sehingga mudah dalam melakukan pengerjaan
- Mengerjakan finalisasi program jauh-jauh hari sehingga menghindari terlewatnya kasus test case dari sebuah masalah

Kepada Asisten

- Memastikan spesifikasi lebih jelas sehingga mengurangi jumlah revisi yang harus diterapkan dari QnA
- Memberikan contoh format error handling yang diharapkan sehingga implementasi recovery mode dapat lebih konsisten juga terarah.
- Menyediakan lebih banyak testcase referensi terutama untuk kasus ambigu dan kompleks sehingga proses pengujian lebih mendalam.

LAMPIRAN

Link Github

<https://github.com/Nusss0/CNA-Tubes-IF2224-2026>

Pembagian Tugas

13524020	25%
13524080	25%
13524104	25%
13524106	25%

REFERENSI

- Arpeggio. "Parse trees." *Arpeggio Documentation*,
https://textx.github.io/Arpeggio/latest/parse_trees/. Accessed 6 May 2026.
- Hopcroft, John E., et al. *Introduction to Automata Theory, Languages, and Computation*. Pearson/Addison Wesley, 2007. Accessed 5 May 2026.
- "Parsing - Introduction to Parsers." *GeeksforGeeks*, 24 April 2026,
<https://www.geeksforgeeks.org/compiler-design/introduction-of-parsing-ambiguity-and-parsers-set-1/>. Accessed 5 May 2026.
- "Pumping Lemma in Theory of Computation." *GeeksforGeeks*, 13 April 2026,
<https://www.geeksforgeeks.org/theory-of-computation/pumping-lemma-in-theory-of-computation/>. Accessed 5 May 2026.
- "Recursive Descent Parser." *GeeksforGeeks*, 11 July 2025,
<https://www.geeksforgeeks.org/compiler-design/recursive-descent-parser/>. Accessed 5 May 2026.
- "Various Properties of context free languages (CFL)." *GeeksforGeeks*, 12 July 2025,
<https://www.geeksforgeeks.org/theory-of-computation/various-properties-of-context-free-languages-cfl/>. Accessed 5 May 2026.
- "What is Context-Free Grammar?" *GeeksforGeeks*, 23 July 2025,
<https://www.geeksforgeeks.org/theory-of-computation/what-is-context-free-grammar/>. Accessed 5 May 2026.